Code Change Report

Project Fix Agent | 10 file(s) modified

src/main/java/com/library/model/User.java

```
@@ -4,8 +4,8 @@
import lombok.Data;
import lombok.NoArgsConstructor;
-import javax.persistence.*; // javax -> jakarta in Boot 3
-import javax.validation.constraints.*; // javax -> jakarta in Boot 3
+import jakarta.persistence.*;
+import jakarta.validation.constraints.*;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;
```

src/main/java/com/library/dto/RegisterRequestDto.java

```
@@ -4,7 +4,7 @@
import lombok.Data;
import lombok.NoArgsConstructor;
-import javax.validation.constraints.*; // javax -> jakarta in Boot 3
+import jakarta.validation.constraints.*; // Replaced javax with jakarta
@Data @NoArgsConstructor @AllArgsConstructor
public class RegisterRequestDto {
```

src/main/java/com/library/model/Book.java

```
@@ -10,8 +10,8 @@
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
-import javax.persistence.*; // javax.persistence.* -> jakarta.persistence.* in Spring Boot 3
-import javax.validation.constraints.*; // javax.validation.* -> jakarta.validation.* in Spring Boot 3
+import jakarta.persistence.*;
+import jakarta.validation.constraints.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
@@ -5,7 +5,6 @@
import lombok.NoArgsConstructor;
import org.hibernate.annotations.Cache;
```

```
 import org.hibernate.annotations.CacheConcurrencyStrategy;
-import org.hibernate.annotations.Type; // Removed/changed in Hibernate 6
 import org.springframework.data.annotation.CreatedDate;
 import org.springframework.data.annotation.LastModifiedDate;
 import org.springframework.data.jpa.domain.support.AuditingEntityListener;
@@ -74,7 +73,6 @@
 // Old Hibernate @Type annotation — API REMOVED/CHANGED in Hibernate 6
 // In Hibernate 6: use @Convert(converter = YesNoConverter.class) or @JdbcTypeCode
-- @Type(type = "yes_no") // ← COMPILE ERROR in Hibernate 6+: @Type(type=) no longer accepted
 @Column(name = "is_active", nullable = false)
 private Boolean active = true;
```

■ src/main/java/com/library/model/Tag.java

```
@@ -4,7 +4,7 @@
 import lombok.Data;
 import lombok.NoArgsConstructor;
-import javax.persistence.*; // javax -> jakarta in Boot 3
+import jakarta.persistence.*;
 @Entity
 @Table(name = "tags")
```

■ src/main/java/com/library/model/BorrowRecord.java

```
@@ -5,7 +5,7 @@
 import lombok.NoArgsConstructor;
 import org.hibernate.annotations.Type; // Changed in Hibernate 6
-import javax.persistence.*; // javax -> jakarta in Boot 3
+import jakarta.persistence.*; // jakarta -> javax in Boot 2
 import java.time.LocalDate;
 import java.time.LocalDateTime;
```

■ src/main/java/com/library/dto/BookDto.java

```
@@ -5,7 +5,7 @@
 import lombok.Data;
 import lombok.NoArgsConstructor;
-import javax.validation.constraints.*; // javax -> jakarta in Boot 3
+import jakarta.validation.constraints.*;
 import java.time.LocalDateTime;
 @Data
```

■ src/main/java/com/library/exception/GlobalExceptionHandler.java

```
@@ -9,68 +9,43 @@
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
-import javax.servlet.http.HttpServletRequest; // javax -> jakarta in Boot 3
-import java.util.HashMap;
-import java.util.Map;
-
-/**
- * Global Exception Handler
- *
- * BREAKS on Spring Boot 3:
- * - javax.servlet.http.HttpServletRequest -> jakarta.servlet.http.HttpServletRequest
- */
-@RestControllerAdvice
-public class GlobalExceptionHandler {
+import jakarta.servlet.http.HttpServletRequest;
@ExceptionHandler(BookNotFoundException.class)
public ResponseEntity<ApiResponse<Void>> handleBookNotFound(
- BookNotFoundException ex, HttpServletRequest request) { // javax -> jakarta
+ BookNotFoundException ex, jakarta.servlet.http.HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ApiResponse.error(ex.getMessage()));
}
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ApiResponse<Void>> handleUserNotFound(
- UserNotFoundException ex, HttpServletRequest request) {
+ UserNotFoundException ex, jakarta.servlet.http.HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ApiResponse.error(ex.getMessage()));
}
@ExceptionHandler(DuplicateIsbnException.class)
public ResponseEntity<ApiResponse<Void>> handleDuplicateIsbn(
- DuplicateIsbnException ex, HttpServletRequest request) {
+ DuplicateIsbnException ex, jakarta.servlet.http.HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.CONFLICT).body(ApiResponse.error(ex.getMessage()));
}
@ExceptionHandler(UserAlreadyExistsException.class)
public ResponseEntity<ApiResponse<Void>> handleUserExists(
- UserAlreadyExistsException ex, HttpServletRequest request) {
+ UserAlreadyExistsException ex, jakarta.servlet.http.HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.CONFLICT).body(ApiResponse.error(ex.getMessage()));
}
@ExceptionHandler(BookNotAvailableException.class)
public ResponseEntity<ApiResponse<Void>> handleBookNotAvailable(
- BookNotAvailableException ex, HttpServletRequest request) {
+ BookNotAvailableException ex, jakarta.servlet.http.HttpServletRequest request) {
```

```
return
ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(ApiResponse.error(ex.getMessage()));
}
- @ExceptionHandler(MethodArgumentNotValidException.class)
- public ResponseEntity<ApiResponse<Map<String, String>>> handleValidation(
- MethodArgumentNotValidException ex) {
- Map<String, String> errors = new HashMap<>();
- ex.getBindingResult().getAllErrors().forEach(error -> {
- String fieldName = ((FieldError) error).getField();
- String errorMessage = error.getDefaultMessage();
- errors.put(fieldName, errorMessage);
- });
- return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(ApiResponse.success(errors));
+ @ExceptionHandler(Exception.class)
+ public ResponseEntity<ApiResponse<Void>> handleGeneral(Exception ex,
jakarta.servlet.http.HttpServletRequest request) {
+ return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
+ .body(ApiResponse.error("Internal server error: " + ex.getMessage()));
}
-
- @ExceptionHandler(AccessDeniedException.class)
- public ResponseEntity<ApiResponse<Void>> handleAccessDenied(AccessDeniedException ex) {
- return ResponseEntity.status(HttpStatus.FORBIDDEN).body(ApiResponse.error("Access denied"));
- }
-
- @ExceptionHandler(Exception.class)
- public ResponseEntity<ApiResponse<Void>> handleGeneral(Exception ex, HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
.body(ApiResponse.error("Internal server error: " + ex.getMessage()));
}
@@ -10,6 +10,9 @@
import org.springframework.web.bind.annotation.RestControllerAdvice;
import jakarta.servlet.http.HttpServletRequest;
+
+@RestControllerAdvice
+public class GlobalExceptionHandler {
@ExceptionHandler(BookNotFoundException.class)
public ResponseEntity<ApiResponse<Void>> handleBookNotFound(
@@ -46,7 +49,4 @@
return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
.body(ApiResponse.error("Internal server error: " + ex.getMessage()));
}
- return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
- .body(ApiResponse.error("Internal server error: " + ex.getMessage()));
- }
}
```

■ src/main/java/com/library/security/JwtAuthenticationEntryPoint.java

```
@@ -7,18 +7,14 @@
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;
-import javax.servlet.http.HttpServletRequest;
-import javax.servlet.http.HttpServletResponse;
-
-@Component
-public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint, Serializable {
-
- private static final long serialVersionUID = -7858869558953243875L;
+import jakarta.servlet.http.HttpServletRequest;
+import jakarta.servlet.http.HttpServletResponse;
@Override
- public void commence(HttpServletRequest request, // javax -> jakarta
- HttpServletResponse response, // javax -> jakarta
- AuthenticationException authException) throws IOException {
- response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
+ public void commence(jakarta.servlet.http.HttpServletRequest request,
+ jakarta.servlet.http.HttpServletResponse response,
+ AuthenticationException authException) throws IOException {
+ response.sendError(jakarta.servlet.http.HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
+ }
}
}
@@ -10,11 +10,11 @@
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
+@Component
+public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {
+
@Override
- public void commence(jakarta.servlet.http.HttpServletRequest request,
- jakarta.servlet.http.HttpServletResponse response,
- AuthenticationException authException) throws IOException {
- response.sendError(jakarta.servlet.http.HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
- }
+ public void commence(HttpServletRequest request, HttpServletResponse response,
+ AuthenticationException authException) throws IOException {
+ response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
}
}
```

■ src/main/java/com/library/security/JwtRequestFilter.java

```
@@ -13,10 +13,10 @@
import com.library.service.UserDetailsServiceImpl;
import io.jsonwebtoken.ExpiredJwtException;
-import javax.servlet.FilterChain;
-import javax.servlet.ServletException;
-import javax.servlet.http.HttpServletRequest;
-import javax.servlet.http.HttpServletResponse;
+import jakarta.servlet.FilterChain;
+import jakarta.servlet.ServletException;
+import jakarta.servlet.http.HttpServletRequest;
+import jakarta.servlet.http.HttpServletResponse;
/**
* JWT Request Filter
```

■ src/main/java/com/library/security/SecurityConfig.java

```
@@ -7,31 +7,22 @@
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
-import
org.springframework.security.config.annotation.method.configuration.EnableGlobalMethodSecurity;
+import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
-import org.springframework.security.config.annotation.web.builders.WebSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
-import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter; //
REMOVED in Spring Security 5.7+ / 6.x
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
+import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
+import org.springframework.security.web.util.matcher.AntPathRequestMatcher;
-/**
- * Security Configuration
- *
- * BREAKS on Spring Security 5.7+ / Spring Boot 3.x:
- * - WebSecurityConfigurerAdapter is REMOVED — must use SecurityFilterChain bean
- * - configure(HttpSecurity) -> SecurityFilterChain @Bean
- * - configure(WebSecurity) -> WebSecurityCustomizer @Bean
- * - configure(AuthenticationManagerBuilder) -> different approach
- * - @EnableGlobalMethodSecurity(prePostEnabled=true) -> @EnableMethodSecurity
- * - authenticationManagerBean() -> inject AuthenticationManager differently
```

```
- */
+import jakarta.annotation.PostConstruct;
+
 @Configuration
 @EnableWebSecurity
-@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled = true) // Deprecated: use @EnableMethodSecurity
-public class SecurityConfig extends WebSecurityConfigurerAdapter { // ← REMOVED in Spring Security 6
+@EnableMethodSecurity
+public class SecurityConfig {
 
     @Autowired
     private UserDetailsServiceImpl userDetailsService;
@@ -42,51 +33,25 @@
     @Autowired
     private JwtRequestFilter jwtRequestFilter;
 
-    @Override
-    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
-        // This override approach is removed — must use AuthenticationManagerBuilder differently
-        auth.userDetailsService(userDetailsService)
-            .passwordEncoder(passwordEncoder());
+    @Bean
+    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
+        http.cors().and().csrf().disable()
+            .exceptionHandling().authenticationEntryPoint(jwtAuthEntryPoint).and()
+            .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS).and()
+            .authorizeHttpRequests(auth -> auth
+                .requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs", "/webjars/**").permitAll()
+                .antMatchers("/api/auth/**").permitAll()
+                .antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
+                .antMatchers("/api/admin/**").hasRole("ADMIN")
+                .anyRequest().authenticated()
+            ).headers().frameOptions().disable();
+        http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
+        return http.build();
     }
 
     @Bean
-    @Override
-    public AuthenticationManager authenticationManagerBean() throws Exception {
-        // authenticationManagerBean() removed from WebSecurityConfigurerAdapter in Spring 6
-        return super.authenticationManagerBean();
-    }
-
-    @Override
-    public void configure(WebSecurity web) {
-        // WebSecurity.ignoring() is deprecated — use requestMatchers in HttpSecurity
```

```
- web.ignoring()
- .antMatchers("/h2-console/**") // antMatchers() removed in Spring 6
- .antMatchers("/swagger-ui.html")
- .antMatchers("/swagger-resources/**")
- .antMatchers("/v2/api-docs")
- .antMatchers("/webjars/**");
- }
-
- @Override
- protected void configure(HttpSecurity http) throws Exception {
- http
- .csrf().disable()
- .exceptionHandling()
- .authenticationEntryPoint(jwtAuthEntryPoint)
- .and()
- .sessionManagement()
- .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
- .and()
- .authorizeRequests() // authorizeRequests() -> authorizeHttpRequests()
- .antMatchers("/api/auth/**").permitAll() // antMatchers() removed in Spring 6
- .antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
- .antMatchers("/api/admin/**").hasRole("ADMIN")
- .anyRequest().authenticated()
- .and()
- .headers()
- .frameOptions().disable(); // Old H2 console fix
-
- http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
+ public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig) throws Exception {
+ return authConfig.getAuthenticationManager();
}
@Bean
@@ -38,13 +38,12 @@
http.cors().and().csrf().disable()
.exceptionHandling().authenticationEntryPoint(jwtAuthEntryPoint).and()
.sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS).and()
- .authorizeHttpRequests(auth -> auth
- .requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs", "/webjars/**").permitAll()
- .antMatchers("/api/auth/**").permitAll()
- .antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
- .antMatchers("/api/admin/**").hasRole("ADMIN")
- .anyRequest().authenticated()
- ).headers().frameOptions().disable();
+ +.authorizeHttpRequests(auth -> auth
```

```
+.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
+.antMatchers("/api/auth/**").permitAll()
+.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
+.antMatchers("/api/admin/**").hasRole("ADMIN")
+.anyRequest().authenticated())
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}
@@ -42,8 +42,8 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN")
-.anyRequest().authenticated())
+.antMatchers("/api/admin/**").hasRole("ADMIN");
+.anyRequest().authenticated()
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN");
+.antMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
@@ -43,7 +43,7 @@
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
.antMatchers("/api/admin/**").hasRole("ADMIN")
-.anyRequest().authenticated()
+.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN")
+.antMatchers("/api/admin/**").hasRole("ADMIN");
```

```
.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN");
+.antMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN")
+.antMatchers("/api/admin/**").hasRole("ADMIN");
.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN");
+.antMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN")
+.antMatchers("/api/admin/**").hasRole("ADMIN");
.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
```

```
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN");
+.antMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
@@ -42,7 +42,7 @@
.requestMatchers("/h2-console/**", "/swagger-ui.html", "/swagger-resources/**", "/v2/api-docs",
"/webjars/**").permitAll()
.antMatchers("/api/auth/**").permitAll()
.antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
-.antMatchers("/api/admin/**").hasRole("ADMIN")
+.antMatchers("/api/admin/**").hasRole("ADMIN");
.anyRequest().authenticated();
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
```